Efficient Skin Lesion Classification via DenseNet169 and PyTorch

Madhav Sharn, 22bcs050, Ansh Singh, 23bcs018

***Abstract*—This report presents a deep learning-based skin lesion classification system utilizing the DenseNet169 architecture implemented in PyTorch. The early detection of malignant skin lesions is clinically vital for improving patient survival rates, but manual dermatoscopic diagnosis is prone to subjective error. Our automated system leverages transfer learning by initializing a DenseNet169 model with pretrained ImageNet weights to classify medical images from the MILK10k dataset into 11 distinct disease categories. Utilizing robust stratified sampling and real-time data augmentation, the evaluated model achieved an overall test accuracy of 61.32% and a weighted ROC-AUC of 84.00%. The substantial discrepancy between the weighted metrics and the macro F1-score (21.15%) reveals the profound impact of dataset class imbalance. This report outlines the architecture, pipeline, and results, demonstrating the potential and challenges of automated diagnosis on imbalanced medical datasets.**

***Index Terms*—Deep Learning, Skin Lesion, DenseNet169, Transfer Learning, Image Classification, Convolutional Neural Networks.**

I. Introduction

SKIN lesions refer to areas of skin tissue that exhibit abnormal growth or appearance compared to the surrounding tissue [1]. Among various types, melanoma is recognized as the most dangerous type of skin cancer, responsible for a vast majority of skin cancer-related fatalities globally [2], [3]. Early detection is clinically imperative; when melanoma is diagnosed initially, the survival rate exceeds 95%, whereas late-stage discovery drops the rate to approximately 13% [4].

Currently, visual examination remains the standard clinical procedure. Dermatologists frequently rely on the standard 'ABCD' protocol to assess lesions [4]. While dermoscopy enhances visibility, the diagnostic process remains inherently subjective, time-consuming, and highly dependent on clinician expertise.

To manage the rising volume of skin disease cases, automated diagnostic tools utilizing deep learning and Convolutional Neural Networks (CNNs) have emerged to automate hierarchical feature extraction [5]. Previous studies have evaluated multiple big data-based CNN models on large datasets, demonstrating that networks pre-trained on ImageNet significantly outperform models trained from scratch [4], [6]. Ensemble learning approaches combining Inception and DenseNet architectures have also achieved remarkable accuracies [7].

This session implements a robust transfer learning approach utilizing the densely connected convolutional network architecture, DenseNet169 [8], to classify skin lesions into 11 distinct clinical categories, evaluating both its structural advantages and the impact of severe class imbalances on performance metrics.

II. Materials and Methods

A. Theoretical Background

Convolutional Neural Networks extract spatial features by sliding a learnable kernel across the input image. Mathematically, the output of a convolution operation with a Rectified Linear Unit (ReLU) activation function is expressed as:

$$y_i = \max\left(0, \sum_j W_{ij} \otimes x_j + b_i\right) \tag{1}$$

where W_{ij} represents the convolution kernel, x_j is the input feature map, b_i is the bias term, and $\otimes$ denotes the discrete convolution operation [7].

Dense Convolutional Networks (DenseNets) introduce a denser connectivity pattern. Within a "dense block," each layer receives the feature maps of all preceding layers as its input:

$$x_\ell = H_\ell([x_0, x_1, \ldots, x_{\ell-1}]) \tag{2}$$

where $[x_0, \ldots, x_{\ell-1}]$ refers to the concatenation of the feature maps, ensuring maximum information flow [8].

B. Dataset and Preprocessing

This study utilized the MILK10k Skin Lesion Dataset, comprising 5,240 usable labeled images across 11 clinical classes (AKIEC, BCC, BEN_OTH, BKL, DF, INF, MAL_OTH, MEL, NV, SCCKA, and VASC).

All images were standardized to 224×224 pixels. To mitigate overfitting, a real-time data augmentation pipeline was implemented using the `torchvision.transforms` module, applying random horizontal flips (p=0.5), vertical flips (p=0.3), rotations up to 20 degrees, and color jittering. Images were normalized utilizing standard ImageNet statistical values.

C. Network and Training Environment

The system utilized the DenseNet169 architecture initialized with ImageNet pre-trained weights. The initial feature extraction layers were frozen to reduce computational overhead. The original classifier layer was replaced with a custom sequential fully-connected block: a linear layer mapping 1664 in-features to 512 hidden units, a ReLU activation, a Dropout layer (0.4 probability), and a final output layer mapping to the 11 target classes.

The model was trained on Google Colab utilizing an NVIDIA Tesla T4 GPU. Data was partitioned using stratified sampling (70% training, 15% validation, 15% testing). The Adam optimizer was used with a weight decay of 1×10^{-4} and a `ReduceLROnPlateau` scheduler for 15 epochs with a batch size of 32.

III. Results

Over the 15-epoch training cycle, the model achieved its highest validation accuracy of 62.60%. When evaluated against the unseen test data, the model recorded an overall classification accuracy of 61.32%.

Because the dataset is heavily imbalanced, overall accuracy is a misleading metric. A comprehensive breakdown of the evaluation metrics is detailed in Table I.

TABLE I
Overall Performance Metrics on Test Set

Metric	Macro Average (%)	Weighted Average (%)
Accuracy	**61.32**	
Precision	26.85	54.73
Recall	21.57	61.32
F1-Score	21.15	54.43
ROC-AUC	82.75	84.00

The confusion matrix (which maps predicted versus actual classes) indicated that the model effectively predicts dominant classes but routinely misclassifies rare pathologies. The disparity between the macro and weighted metrics clearly illustrates this limitation in real-world application.

Fig. 1. Confusion matrix representing the model's prediction mapping across the 11 MILK10k skin lesion classes.

IV. Discussion and Summary

The experimental results validate the structural capabilities of DenseNet169 as a feature extractor, as theorized in Equation 2, but expose severe limitations imposed by imbalanced medical data.

While the model achieved an overall test accuracy of 61.32%, the significant disparity between the weighted F1-score (54.43%) and the macro F1-score (21.15%) highlights critical diagnostic flaws. The network's gradient updates were dominated by majority classes (BCC and NV), failing to construct robust decision boundaries for minority classes (BEN_OTH, DF, INF, and VASC).

Despite poor macro-level metrics, the high ROC-AUC scores (82.75% Macro, 84.00% Weighted) suggest the underlying probability distributions are learning meaningful spatial features, even if the rigid class prediction struggles with minority thresholds. In conclusion, while transfer learning with CNNs is highly effective for medical imaging, overall predictive performance heavily relies on balanced dataset distributions.

V. Future Scope

To transition this automated system toward clinical reliability and improve upon the limitations discovered during testing, future work should prioritize the following strategies:

- **Addressing Class Imbalance:** Implementing class-weighted Cross-Entropy loss or utilizing a `WeightedRandomSampler` during the dataloading phase to heavily penalize majority class dominance and force the network to learn minority features.
- **Deep Fine-Tuning:** Gradually unfreezing the deeper dense blocks of the DenseNet169 architecture. This will allow the network to adapt its mid-level visual features specifically to dermatoscopic textures rather than relying purely on broad ImageNet representations.
- **Multimodal Integration:** Fusing the CNN's extracted visual image vectors with patient clinical metadata (such as age, sex, and physical lesion location) prior to the final classification layer to provide a more holistic diagnostic prediction.
- **Explainable AI (XAI):** Integrating techniques such as Grad-CAM to generate visual heatmaps of the model's spatial attention, ensuring the decision-making process is transparent, interpretable, and trustworthy for medical professionals.

Acknowledgment

The authors would like to thank the Deep Learning Project Team and the faculty of the Biomedical Instrumentation lab for their guidance and access to computational resources.

References

[1] J. L. Longe, *The Gale Encyclopedia of Medicine*. Gale, Cengage Learning, 2015.
[2] Mayo Foundation for Medical Education and Research, "How skin cancer develops," 2017.
[3] American Cancer Society, "Melanoma Skin Cancer Statistics," Atlanta, 2019.
[4] P. Naronglerdrit and I. Mporas, "Evaluation of Big Data based CNN Models in Classification of Skin Lesions with Melanoma," University of Hertfordshire Research Archive, Hatfield, UK.
[5] A. Esteva *et al.*, "Dermatologist-level classification of skin cancer with deep neural networks," *Nature*, vol. 542, p. 115, 2017.
[6] P. Tschandl, C. Rosendahl, and H. Kittler, "The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions," *Scientific Data*, vol. 5, no. 1, 2018.
[7] R. A. Pratiwi, S. Nurmaini, D. P. Rini, M. N. Rachmatullah, and A. Darmawahyuni, "Deep ensemble learning for skin lesions classification with convolutional neural network," *IAES International Journal of Artificial Intelligence (IJ-AI)*, vol. 10, no. 3, pp. 563-570, 2021.
[8] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognition, CVPR 2017*, 2017, pp. 2261-2269.

Appendix A
Weighted Random Sampler Implementation

To address the class imbalance observed in the results, the following PyTorch implementation was developed as a proposed solution for future data-loading pipelines. This ensures the model sees rare classes as frequently as common ones.

```
import torch
from torch.utils.data import WeightedRandomSampler,
DataLoader
import numpy as np

# 1. Count total samples per class
class_counts = np.bincount(train_dataset.targets)

# 2. Calculate weight (inversely proportional)
class_weights = 1.0 / class_counts

# 3. Assign weight to every training sample
```

```
sample_weights = np.array(
    [class_weights[t] for t in train_dataset.targets]
)
sample_weights = torch.from_numpy(sample_weights)
.double()

# 4. Initialize the WeightedRandomSampler
sampler = WeightedRandomSampler(
    weights=sample_weights,
    num_samples=len(sample_weights),
    replacement=True
)

# 5. Pass sampler to DataLoader
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    sampler=sampler,
    shuffle=False
)
```